Pseudoklasy :not(), :where(), :is(), :has()

Wprowadzenie

CSS rozwija się nie tylko jako język opisu wyglądu, ale także jako coraz bardziej ekspresyjne narzędzie **selekcji elementów** w drzewie DOM. Szczególną rolę odgrywają tu pseudoklasy funkcyjne, czyli takie, które przyjmują argument w nawiasie. Do najważniejszych należą:

- :not()
- :where()
- :is()
- :has()

Łączy je to, że pozwalają konstruować bardziej elastyczne, krótsze i łatwiejsze w utrzymaniu selektory. Różnią się jednak:

- logiką działania,
- wpływem na precyzję (specyficzność),
- kierunkiem selekcji,
- typowymi zastosowaniami projektowymi.

:not() — pseudoklasa negacji

Definicja

Pseudoklasa `:not()` służy do **wykluczenia** elementów pasujących do określonego selektora.

Ogólna postać

```
selektor:not(argument)
```

Oznacza to: wybierz elementy pasujące do `selektor`, ale **niepasujące** do `argument`.

Przykład podstawowy

```
p:not(.intro) {
    color: #333;
}
```

Interpretacja:

- wybierz wszystkie akapity `<p>`,
- z wyjątkiem tych, które mają klasę `.intro`.

Przykład rozbudowany

```
button:not(:disabled):not(.secondary) {
    background: royalblue;
    color: white;
}
```

Tu selektor wybiera:

- przyciski `button`,
- które nie są wyłączone,
- i nie należą do klasy `.secondary`.

To klasyczny przykład filtrowania elementów interfejsu.

Zastosowania praktyczne

a) stylowanie wszystkich elementów z wyjątkiem jednego typu

```
nav a:not(.active) {
    opacity: 0.7;
}
```

b) unikanie ręcznego nadawania klas

```
ul.menu li:not(:last-child) {
    margin-right: 1rem;
}
```

c) selekcja pól formularza poza określonym wariantem

```
input:not([type="checkbox"]) {
    width: 100%;
}
```

Uwagi teoretyczne

W starszych etapach rozwoju CSS `:not()` było bardziej ograniczone. W nowoczesnym CSS argument może być znacznie bogatszy i obejmować bardziej złożone selektory.

Ważna uwaga

`:not()` **nie wybiera samodzielnie elementu innego typu**, lecz działa jako filtr negacyjny na już wskazanym zbiorze.

Precyzja (specyficzność) `:not()`

Sama pseudoklasa `:not()` **nie dodaje własnej precyzji (specyficzności)**, ale przejmuje precyzję (specyficzność) argumentu.

Przykład:

```
div:not(.card)
```

Precyzja (specyficzność) pochodzi od:

- div
- .card

czyli selektor nie jest „lekki”, mimo że wygląda niewinnie.

:is() — grupowanie alternatyw

Definicja

Pseudoklasa `:is()` pozwala zapisać **alternatywne selektory** w jednej zwartej formie.

Ogólna postać

```
:is(selektor1, selektor2, selektor3)
```

Znaczenie:

- wybierz elementy pasujące do **któregokolwiek** z podanych selektorów.

Przykład podstawowy

```
:is(h1, h2, h3) {
    line-height: 1.2;
}
```

To odpowiednik:

```
h1, h2, h3 {
    line-height: 1.2;
}
```

Przykład praktyczny

```
article :is(h2, h3, h4) {
    color: #1f3a5f;
}
```

Interpretacja:

- w obrębie `article`
- wybierz wszystkie `h2`, `h3`, `h4`.

Zastosowania

a) skracanie złożonych selektorów

Zamiast:

```
main h1,
main h2,
main h3 {
    margin-top: 0;
}
```

piszemy:

```
main :is(h1, h2, h3) {
    margin-top: 0;
}
```

b) wspólne stylowanie wariantów komponentów

```
.button:is(.primary, .danger, .success) {
    color: white;
}
```

c) uproszczenie selekcji w dużych systemach projektowych

```
.card :is(a, button, input, select) {
    font: inherit;
}
```

Precyzja (specyficzność) `:is()`

`:is()` przejmuje **najwyższą precyzja (specyficzność)** spośród selektorów wpisanych jako argumenty.

Przykład:

```
:is(div, .box, #hero)
```

Precyzja (specyficzność) całego argumentu będzie taka jak `#hero`, bo to najsilniejszy składnik.

To ma duże znaczenie praktyczne: `:is()` skraca kod, ale potrafi także nieoczekiwanie podnieść siłę selektora.

:where() — grupowanie bez kosztu precyzji (specyficzności)

Definicja

`:where()` działa logicznie podobnie do `:is()`, ale z jedną kluczową różnicą: **zawsze ma zerową precyzja (specyficzność)**.

Ogólna postać

```
:where(selektor1, selektor2, selektor3)
```

Przykład podstawowy

```
:where(h1, h2, h3) {
    margin-bottom: 0.5em;
}
```

Działa podobnie jak `:is()`, ale nie „obciąża” kaskady.

Najważniejsza cecha

`:where()` jest idealne tam, gdzie chcemy:

- pisać selektory wygodnie,
- ale nie chcemy zwiększać ich siły,
- żeby późniejsze nadpisanie stylów było łatwe.

Przykład dydaktyczny

```
:where(article, section, aside) a {
    text-decoration-thickness: 2px;
}
```

Tak zapisany selektor jest dużo bezpieczniejszy dla dalszego rozszerzania niż analogiczny zapis oparty o `:is()`.

Zastosowania

a) style bazowe / reset / design system

```
:where(ul, ol) {
    padding-left: 1.2rem;
}
```

b) lekkie stylowanie komponentów

```
.card :where(h2, h3, p) {
    margin-block: 0;
}
```

c) tworzenie „miękkich” selektorów

```
:where(nav ul li a) {
```

```
        text-decoration: none;
    }
```

Taki zapis daje styl bazowy, który łatwo potem nadpisać.

Precyzja (specyficzność) `:where()`

To najważniejsza różnica względem `:is()`:

- `:is()` → bierze najwyższą precyzja (specyficzność) argumentu
- `:where()` → ma zawsze precyzja (specyficzność) 0

Nawet jeśli w środku wpiszemy coś „mocnego”, np. `#id`, `:where()` nie zwiększy specyficzności selektora.

To czyni ją szczególnie wartościową w nowoczesnej architekturze CSS.

:has() — pseudoklasa relacyjna

Definicja

`:has()` wybiera element na podstawie tego, co zawiera lub z czym jest relacyjnie powiązany.

To pseudoklasa **przełomowa**, bo umożliwia w CSS coś, co dawniej wymagało JavaScript: **stylowanie rodzica w zależności od potomka** albo elementu wcześniejszego na podstawie elementu późniejszego.

Ogólna postać

```
selektor:has(argument)
```

Znaczenie:

- wybierz `selektor`,
- jeśli zawiera lub spełnia relację opisaną w `argument`.

Przykład podstawowy

```
article:has(img) {
    padding-top: 2rem;
}
```

Interpretacja:

- wybierz tylko te `article`,
- które zawierają obrazek `img`.

Przykład formularzowy

```
form:has(input:invalid) {
    border: 2px solid crimson;
}
```

To bardzo ważny przypadek:

- formularz zostanie wyróżniony,
- jeśli zawiera pole niepoprawne.

Przykład z checkboxem

```
label:has(input:checked) {
    font-weight: bold;
}
```

Oznacza:

- wybierz etykietę `label`,
- jeśli zawiera zaznaczony checkbox.

To jeden z najbardziej eleganckich przykładów wykorzystania `:has()`.

Przykład „rodzic zależny od dziecka”

```
.card:has(.error) {
    border-color: red;
}
```

Karta komponentu otrzyma czerwone obramowanie, jeśli wewnątrz wystąpi element `.error`.

Przykład relacji między rodzeństwem

```
h2:has(+ p) {
    margin-bottom: 0.2em;
}
```

Interpretacja:

- wybierz `h2`,
- jeśli bezpośrednio po nim występuje `p`.

To bardzo subtelne i użyteczne w typografii interfejsu.

Zastosowania praktyczne

a) formularze

```
.field:has(input:focus) {
    outline: 2px solid dodgerblue;
}
```

b) karty produktów

```
.product:has(.badge-sale) {
    box-shadow: 0 0 0 2px orange;
}
```

c) menu

```
li:has(> ul) > a::after {
    content: " ▾ ";
}
```

To pozwala dodać wskaźnik rozwijania tylko do tych pozycji menu, które mają podmenu.

Znaczenie teoretyczne

`:has()` jest często określane jako **“parent selector”**, choć technicznie jest czymś więcej: to **pseudoklasa relacyjna**, bo może badać różne relacje strukturalne, nie tylko prostą zawartość potomka.

Precyzja (specyficzność) `:has()`

Podobnie jak `:is()` i `:not()`, `:has()` przejmuje precyzję (specyficzność) argumentu.

Przykład:

```
.card:has(#featured)
```

Taki selektor będzie bardzo silny, bo argument zawiera `#featured`.

Porównanie logiczne

:not()
wybiera elementy, które **nie spełniają** warunku

:is()
wybiera elementy, które spełniają **jeden z wielu** warunków

:where()
działa jak `:is()`, ale nie zwiększa specyficzności

:has()
wybiera element na podstawie tego, co zawiera lub z czym pozostaje w relacji

Porównanie na przykładach

a) negacja

```
input:not([type="checkbox"]) {
    display: block;
}
```

b) alternatywa

```
:is(h1, h2, h3) {
    font-family: Georgia, serif;
}
```

c) alternatywa „lekka”

```
:where(h1, h2, h3) {
    margin-top: 0;
}
```

d) rodzic zależny od dziecka

```
section:has(h2) {
  padding-top: 2rem;
}
```

Tabela porównawcza

Pseudoklasa	Funkcja	Precyzja (specyficzność)	Typowe zastosowanie
`:not()`	negacja	jak argument	wykluczanie elementów
`:is()`	alternatywa	najwyższa z argumentów	skracanie selektorów
`:where()`	alternatywa	0	style bazowe, reset, architektura CSS
`:has()`	relacja / zawartość	jak argument	stylowanie rodzica, logika UI

Zastosowania w projektowaniu interfejsów

Te pseudoklasy mają ogromne znaczenie dla projektowania nowoczesnych interfejsów.

:not()
pozwala filtrować komponenty bez mnożenia klas:

```
.button:not(.disabled):not(.secondary) { ... }
```

:is()
upraszcza kod w systemach design system:

```
.card :is(h2, h3, h4) { ... }
```

:where()
umożliwia tworzenie warstwy stylów bazowych, które łatwo nadpisać:

```
:where(button, input, select, textarea) {
    font: inherit;
}
```

:has()
pozwala budować bardziej reaktywne UI bez JavaScript:

```
.form-group:has(input:invalid) {
    color: crimson;
}
```

Najważniejsze pułapki

1. Mylenie `:is()` Z `:where()`

Logicznie podobne, ale kaskadowo bardzo różne.

2. Nieuwzględnianie precyzji (specyficzności)

`#id` wpisane do `:is()` albo `:has()` może niespodziewanie „wzmocnić” selektor.

3. Nadużywanie `:has()`

Choć bardzo potężne, może prowadzić do zbyt złożonych selektorów i trudniejszego utrzymania kodu.

4. Używanie `:not()` do nadmiernie skomplikowanej logiki

Selekcja negatywna jest użyteczna, ale przy zbyt rozbudowanych warunkach pogarsza czytelność arkusza.

Wnioski

Pseudoklasy `:not()`, `:where()`, `:is()` i `:has()` reprezentują ważny etap rozwoju CSS: przejście od prostego stylowania elementów do **bardziej deklaratywnego modelu opisu relacji i stanów interfejsu**.

Z punktu widzenia teorii i praktyki:

- `:not()` organizuje selekcję przez wykluczenie,
- `:is()` kompresuje logikę alternatyw,
- `:where()` wspiera architekturę stylów o niskiej specyficzności,
- `:has()` otwiera drogę do relacyjnego, niemal „warunkowego” CSS.

W dobrze zaprojektowanym kodzie nie są one ozdobą składni, lecz narzędziami:

- poprawy czytelności,
- redukcji powtórzeń,
- zwiększenia modularności,
- lepszego odwzorowania logiki interfejsu.

Krótki zestaw przykładów końcowych

```
/* 1. wszystkie linki poza przyciskiem */
a:not(.button) {
    text-decoration: underline;
}

/* 2. nagłówki różnych poziomów */
:is(h2, h3, h4) {
    color: #1b2d4f;
}

/* 3. lekkie style bazowe */
:where(ul, ol) {
    padding-left: 1.25rem;
}

/* 4. karta z błędem */
.card:has(.error-message) {
    border: 1px solid crimson;
}
```